Product Recommendation System

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
f1_score, roc_auc_score, confusion_matrix, roc_curve)
pd.set_option('display.width', 90)
pd.set_option('display.max_columns', None)
# ---------- 1. Create and load the recommendation data ----------
rng = np.random.RandomState(42)
n = 20000
df = pd.DataFrame({
'Product_Price': np.round(rng.lognormal(mean=3.8, sigma=1.0, size=n), 2),
'Product_Rating': np.round(rng.uniform(1, 5, size=n), 1),
'Customer_Age': rng.randint(18, 70, size=n),
'Past_Purchases': rng.poisson(3, size=n),
'Time_On_Site_Min': np.round(rng.exponential(scale=8, size=n), 2),
'Is_Discount': rng.binomial(1, 0.30, size=n),
'Is_Same_Category': rng.binomial(1, 0.25, size=n),
'Is_Trending': rng.binomial(1, 0.20, size=n),
})
logit = (-5.5
+ 0.6 * df['Product_Rating']
- 0.35 * np.log(df['Product_Price'])
+ 0.15 * df['Past_Purchases']
+ 0.03 * df['Time_On_Site_Min']
+ 0.9 * df['Is_Discount']
+ 1.6 * df['Is_Same_Category']
+ 0.8 * df['Is_Trending']
- 0.01 * df['Customer_Age'])
prob = 1 / (1 + np.exp(-logit))
df['Is_Purchased'] = rng.binomial(1, prob)
print(df.head())
print("Shape:", df.shape)
# ---------- 2. Univariate analysis ----------
def univariate_analysis(df, column):
    freq = df[column].value_counts()
    mean = df[column].mean()
    median = df[column].median()
    mode = df[column].mode()[0]
    std = df[column].std()
    return freq, mean, median, mode, std
freq, mean, median, mode, std = univariate_analysis(df, 'Product_Price')
print("\nUnivariate Analysis for Product_Price")
print("Frequency (top 5):\n", freq.head())
print("Mean:", mean)
print("Median:", median)
print("Mode:", mode)
print("Standard Deviation:", std)
print("\nClass distribution (0 = Not Purchased, 1 = Purchased):")
print(df['Is_Purchased'].value_counts())
print("Purchase percentage:", df['Is_Purchased'].mean() * 100)
plt.figure(figsize=(10, 4))
df['Is_Purchased'].value_counts().sort_index().plot(kind='bar', color=['tab:blue', 'tab:green'])
plt.title("Not Purchased (0) vs Purchased (1) Recommendations")
plt.xlabel("Class")
plt.ylabel("Number of Recommendations")
plt.xticks(rotation=0)
plt.show()
# ---------- 3. Features and target ----------
features = ['Product_Price', 'Product_Rating', 'Customer_Age', 'Past_Purchases',
'Time_On_Site_Min', 'Is_Discount', 'Is_Same_Category', 'Is_Trending']
X = df[features]
y = df['Is_Purchased']
# ---------- 4. Stratified train-test split ----------
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y)
print("\nTotal, Train, Test:", len(df), len(X_train), len(X_test))
print("Purchased in Train, Test:", y_train.sum(), y_test.sum())
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
# ---------- 5. Logistic regression ----------
log_reg = LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)
```

```
log_reg.fit(X_train_sc, y_train)
pred_lr = log_reg.predict(X_test_sc)
prob_lr = log_reg.predict_proba(X_test_sc)[:, 1]
print("\nLogistic Regression")
print("Coefficients:", log_reg.coef_[0])
print("Intercept:", log_reg.intercept_[0])
print("F1 Score:", f1_score(y_test, pred_lr))
# ---------- 6. Decision tree ----------
tree = DecisionTreeClassifier(max_depth=6, class_weight='balanced', random_state=42)
tree.fit(X_train, y_train)
pred_dt = tree.predict(X_test)
prob_dt = tree.predict_proba(X_test)[:, 1]
print("\nDecision Tree")
print("F1 Score:", f1_score(y_test, pred_dt))
# ---------- 7. Random forest ----------
forest = RandomForestClassifier(n_estimators=100, max_depth=8, class_weight='balanced',
random_state=42, n_jobs=1)
forest.fit(X_train, y_train)
pred_rf = forest.predict(X_test)
prob_rf = forest.predict_proba(X_test)[:, 1]
print("\nRandom Forest")
print("F1 Score:", f1_score(y_test, pred_rf))
print("Feature Importance:")
for name, imp in sorted(zip(features, forest.feature_importances_), key=lambda t: -t[1]):
    print(" ", name, ":", round(imp, 4))
# ---------- 8. Naive baseline ----------
naive_pred = np.zeros(len(y_test), dtype=int)
print("\nNaive baseline accuracy (predict everything as Not Purchased):",
accuracy_score(y_test, naive_pred))
print("Naive baseline recall:", recall_score(y_test, naive_pred))
# ---------- 9. Comparison ----------
results = {}
for name, pred, prob in [("Logistic Regression", pred_lr, prob_lr),
("Decision Tree", pred_dt, prob_dt),
("Random Forest", pred_rf, prob_rf)]:
    results[name] = {
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred, zero_division=0),
        "Recall": recall_score(y_test, pred),
        "F1": f1_score(y_test, pred),
        "ROC_AUC": roc_auc_score(y_test, prob),
    }
results_df = pd.DataFrame(results).T
print("\nModel Comparison")
print(results_df.round(4))
best_name = results_df['F1'].idxmax()
print("Better model:", best_name)
# ---------- 10. Confusion matrix of the best model ----------
best_pred = {"Logistic Regression": pred_lr, "Decision Tree": pred_dt,
"Random Forest": pred_rf}[best_name]
cm = confusion_matrix(y_test, best_pred)
print("\nConfusion Matrix (", best_name, ")")
print(cm)
plt.figure(figsize=(5, 4))
plt.imshow(cm, cmap='Blues')
plt.title("Confusion Matrix - " + best_name)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.xticks([0, 1], ["Not Purchased", "Purchased"])
plt.yticks([0, 1], ["Not Purchased", "Purchased"])
for i in range(2):
    for j in range(2):
        plt.text(j, i, cm[i, j], ha='center', va='center',
        color='white' if cm[i, j] > cm.max() / 2 else 'black')
plt.colorbar()
plt.show()
# ---------- 11. ROC curve ----------
plt.figure(figsize=(10, 4))
for name, prob in [("Logistic Regression", prob_lr), ("Decision Tree", prob_dt),
("Random Forest", prob_rf)]:
    fpr, tpr, _ = roc_curve(y_test, prob)
    plt.plot(fpr, tpr, label=name + " (AUC = " + str(round(roc_auc_score(y_test, prob), 3)) + ")")
plt.plot([0, 1], [0, 1], 'k--', label="Random guess")
plt.title("ROC Curve - Product Recommendation")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()
```

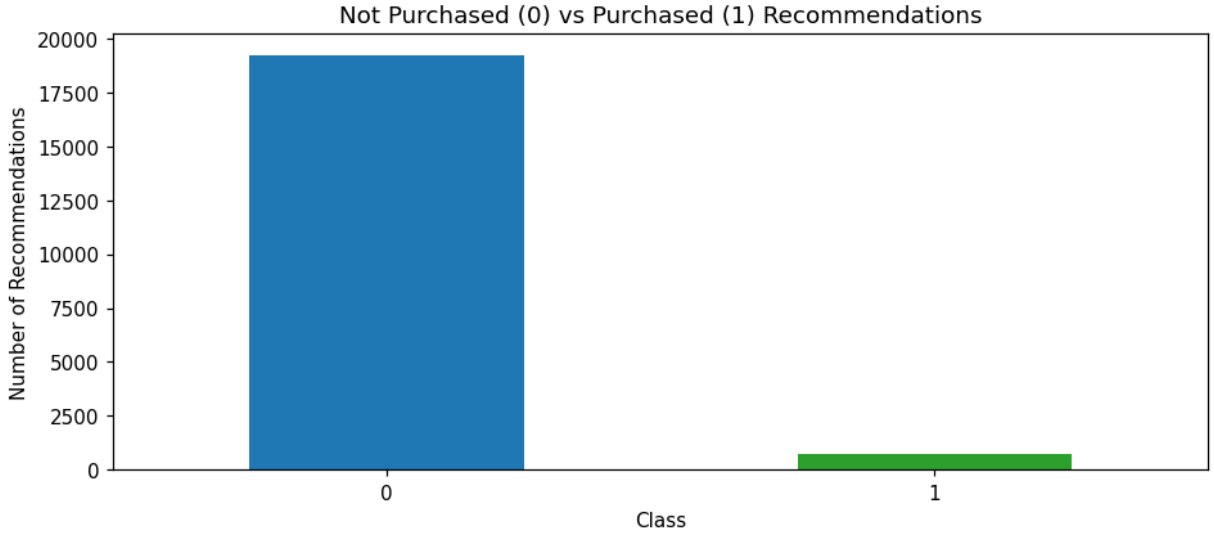
Output

```
   Product_Price  Product_Rating  Customer_Age  Past_Purchases  Time_On_Site_Min  \
0          73.46             1.2            29               7              8.16
1          38.93             4.9            66               3              5.44
2          85.43             4.2            51               4             16.66
3         205.00             4.5            66               3              9.76
4          35.37             3.8            42               2              1.77

   Is_Discount  Is_Same_Category  Is_Trending  Is_Purchased
0            0                 0            0             0
1            0                 0            0             0
2            0                 0            1             0
3            0                 1            0             0
4            0                 1            0             0
Shape: (20000, 9)

Univariate Analysis for Product_Price
Frequency (top 5):
 Product_Price
20.64    10
20.02    10
16.02    10
17.02    10
14.44     9
Name: count, dtype: int64
Mean: 74.40234299999999
Median: 45.05
Mode: 16.02
Standard Deviation: 100.72178827906133

Class distribution (0 = Not Purchased, 1 = Purchased):
Is_Purchased
0    19261
1      739
Name: count, dtype: int64
Purchase percentage: 3.695
```




```
Total, Train, Test: 20000 16000 4000
Purchased in Train, Test: 591 148

Logistic Regression
Coefficients: [-0.36145272  0.65168679 -0.08603245  0.20897046  0.31043925  0.41183431
  0.62857389  0.37552072]
Intercept: -0.6640900027099942
F1 Score: 0.16615384615384615

Decision Tree
F1 Score: 0.14760147601476015

Random Forest
F1 Score: 0.21553090332805072
Feature Importance:
  Product_Rating : 0.2275
  Is_Same_Category : 0.1714
  Product_Price : 0.1673
  Time_On_Site_Min : 0.1509
```

```
  Customer_Age : 0.0975
  Is_Discount : 0.0638
  Past_Purchases : 0.0634
  Is_Trending : 0.0583

Naive baseline accuracy (predict everything as Not Purchased): 0.963
Naive baseline recall: 0.0

Model Comparison
                     Accuracy  Precision  Recall      F1  ROC_AUC
Logistic Regression    0.7290     0.0938  0.7297  0.1662   0.8097
Decision Tree          0.7113     0.0829  0.6757  0.1476   0.7186
Random Forest          0.8762     0.1408  0.4595  0.2155   0.8029
Better model: Random Forest

Confusion Matrix ( Random Forest )
[[3437  415]
 [  80   68]]
```

